Min Number Of Coins For Change

AlgoExpert · Medium · Dynamic Programming

The problem

- Fewest coins summing to n, or -1 if impossible.

Intuition

- Build the fewest coins for every amount from zero up to n.

Brute force vs optimal

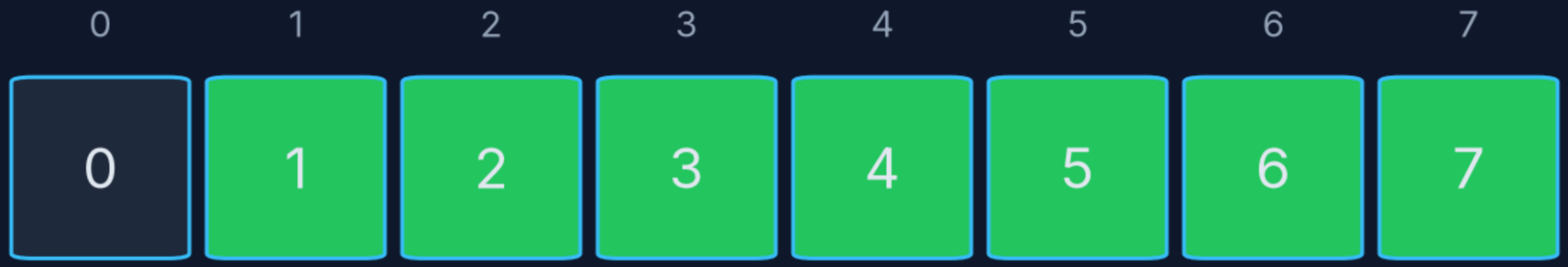
- Greedy (largest coin first) can fail, e.g. coins [1, 3, 4], n = 6
- Recursion over choices → exponential
- Bottom-up DP → O(n · d) time, O(n) space

Approach

- 1. Array of infinities, zero for amount zero
- 2. For each coin, relax amounts from the coin up
- 3. Answer or minus one

Worked example

min coins for each amount 0...7

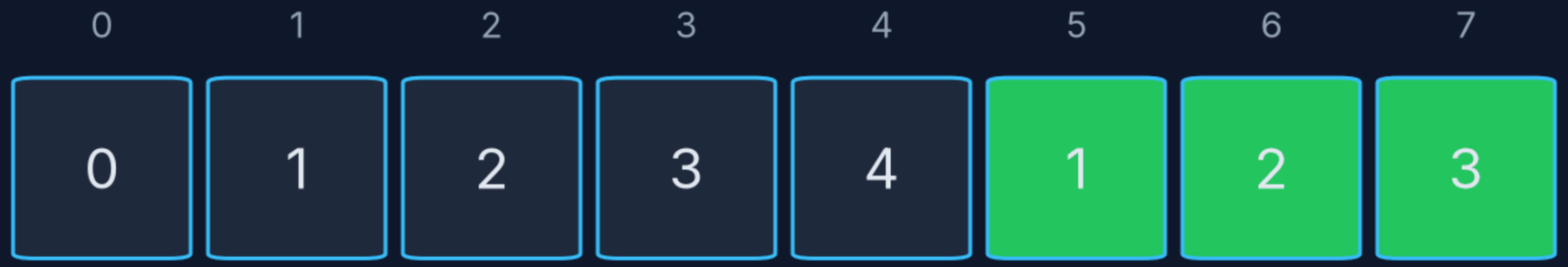



```
after coin 1
```

After coin 1: dp = [0, 1, 2, 3, 4, 5, 6, 7]

Worked example

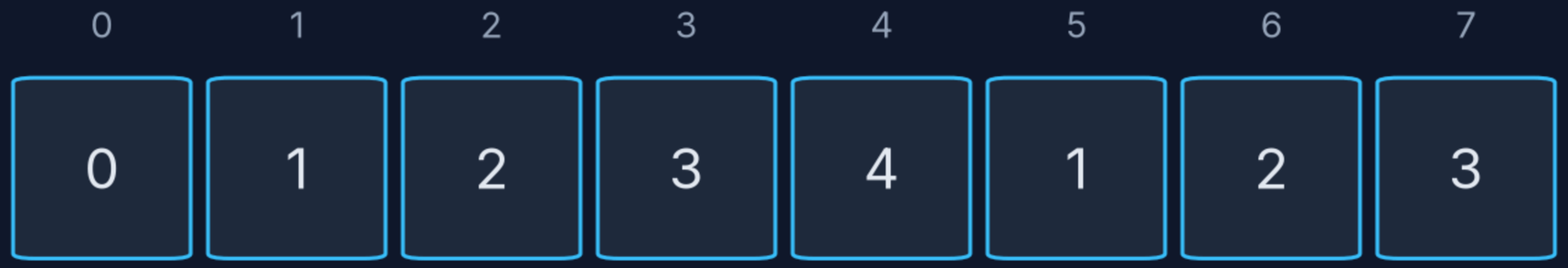
min coins for each amount 0...7




```
after coin 5
```

After coin 5: dp = [0, 1, 2, 3, 4, 1, 2, 3]

Worked example




```
coin 10 > 7: no change
```

Coin 10 is too big

Java solution

```java
public class MinNumberOfCoinsForChange {
    public static int minNumberOfCoinsForChange(int n, int[] denoms) {
        int[] dp = new int[n + 1];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[0] = 0;
        for (int d : denoms)
            for (int a = d; a <= n; a++)
                if (dp[a - d] != Integer.MAX_VALUE) dp[a] = Math.min(dp[a], dp[a - d] + 1);
        return dp[n] == Integer.MAX_VALUE ? -1 : dp[n];
    }
}
```

Complexity

- Time: O(n*d)
- Space: O(n)

Edge cases & pitfalls

- Amount 0 → 0 coins
- Unreachable → −1
- Guard INF + 1 overflow

Interview tips & follow-ups

- Show the greedy counterexample
- Follow-up: return the actual coins (store choices)

Recap

- Build the fewest coins for every amount from zero up to n.
- Complexity: O(n*d)